

Random Forest

Decision Making using Multiple Decision Trees

Nirjhor Datta

Motivation: Why Random Forest?

A single decision tree is easy to understand, but it can have problems.

- It may overfit the training data.
- A small change in data can change the whole tree.
- One tree may make a wrong decision.

Main Idea

Instead of using only one decision tree, Random Forest builds many decision trees and combines their decisions.

Intuition

One weak decision may be wrong, but many independent decisions together are usually more reliable.

What is Random Forest?

Random Forest is a decision-tree-based machine learning model.

Uses

Random Forest can be used for:

- Classification
- Regression
- Feature selection
- Missing value handling

Simple Definition

Random Forest is an ensemble of many decision trees.

Random Forest = Many Decision Trees + Voting/Averaging

Meaning of “Random” and “Forest”

Random

The word **Random** refers to random selection of training examples and random selection of features.

- Random data samples
- Random feature subsets

Forest

The word **Forest** means many decision trees are used together.

- Tree 1
- Tree 2
- Tree 3
- Many more trees

Key Point

Random Forest creates diversity among trees so that all trees do not make the same mistake.

Basic Components of Random Forest

Random Forest mainly uses three ideas:

- 1 **Bootstrapping**
- 2 **Random feature selection**
- 3 **Bagging / majority voting**

For Classification

Each tree predicts a class label. The final class is selected by majority voting.

For Regression

Each tree predicts a numerical value. The final prediction is the average of all tree predictions.

Bootstrapping

Bootstrapping means creating a random dataset by sampling from the original dataset.

Important

Sampling is usually done **with replacement**.

That means the same training example can appear more than once in a bootstrapped dataset.

Example

Original data:

D_1, D_2, D_3, D_4, D_5

One bootstrapped dataset could be:

D_2, D_2, D_5, D_1, D_4

Purpose

Each decision tree gets a slightly different training dataset.

Random Feature Selection

Besides random data sampling, Random Forest also uses random feature selection.

Idea

For each tree, or sometimes for each split, only a random subset of features is considered.

Suppose the original features are:

Outlook, Temperature, Humidity, Wind

One tree may use:

Temperature, Humidity

Another tree may use:

Outlook, Temperature

Another tree may use:

Bagging and Majority Voting

Bagging means combining the outputs of many models trained on bootstrapped datasets.

For Classification

Each tree gives one vote.

Example

For a query sample:

$Tree_1 = \text{Yes}$

$Tree_2 = \text{No}$

$Tree_3 = \text{No}$

Final prediction:

No

because No gets the majority vote.

Random Forest Classification Steps

- 1 Create multiple bootstrapped datasets.
- 2 For each dataset, choose a random subset of features.
- 3 Build one decision tree from each dataset.
- 4 Use Gini impurity or another split criterion to build each tree.
- 5 For a new query, pass it through all trees.
- 6 Combine tree predictions using majority voting.

Final Decision

The class predicted by most trees becomes the Random Forest output.

Example Dataset: Play Tennis

We will use the Play Tennis dataset.

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	No
Day2	Sunny	Hot	High	Strong	No
Day3	Overcast	Hot	High	Weak	Yes
Day4	Rain	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	No
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	No
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	Yes
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	No

Bootstrapped Dataset 1

Suppose Random Forest creates this bootstrapped dataset:

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	Yes
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	No
Day2	Sunny	Hot	High	Strong	No

Now suppose only two features are randomly selected:

Temperature, Humidity

Selected Columns

We build a decision tree using only Temperature and Humidity.

Dataset 1: Selected Features

Day	Temperature	Humidity	Play Tennis
Day10	Mild	Normal	Yes
Day11	Mild	Normal	Yes
Day12	Mild	High	Yes
Day13	Hot	Normal	Yes
Day14	Mild	High	No
Day2	Hot	High	No

Goal

Choose the best root feature between:

Temperature and Humidity

Split Criterion

We use Gini impurity. Lower Gini means better separation.

Reminder: Gini Impurity

For a node with two classes:

$$Gini = 1 - p_{Yes}^2 - p_{No}^2$$

where:

$$p_{Yes} = \frac{\text{Number of Yes samples}}{\text{Total samples}}$$

$$p_{No} = \frac{\text{Number of No samples}}{\text{Total samples}}$$

Interpretation

- Gini = 0 means the node is pure.
- Higher Gini means the node is more mixed.

Dataset 1: Gini for Temperature

Temperature has two values:

$$Temperature \in \{Mild, Hot\}$$

Temperature	Yes	No	Gini
Mild	3	1	$1 - (3/4)^2 - (1/4)^2 = 0.375$
Hot	1	1	$1 - (1/2)^2 - (1/2)^2 = 0.5$

Weighted Gini:

$$Gini(Temperature) = \frac{4}{6}(0.375) + \frac{2}{6}(0.5)$$

$$Gini(Temperature) = 0.417$$

Dataset 1: Gini for Humidity

Humidity has two values:

$$\text{Humidity} \in \{\text{High}, \text{Normal}\}$$

Humidity	Yes	No	Gini
High	1	2	$1 - (1/3)^2 - (2/3)^2 = 0.444$
Normal	3	0	$1 - (3/3)^2 - (0/3)^2 = 0$

Weighted Gini:

$$\text{Gini}(\text{Humidity}) = \frac{3}{6}(0.444) + \frac{3}{6}(0)$$

$$\text{Gini}(\text{Humidity}) = 0.222$$

Dataset 1: Choose Root Node

Compare the two Gini values:

$$Gini(Temperature) = 0.417$$

$$Gini(Humidity) = 0.222$$

Decision

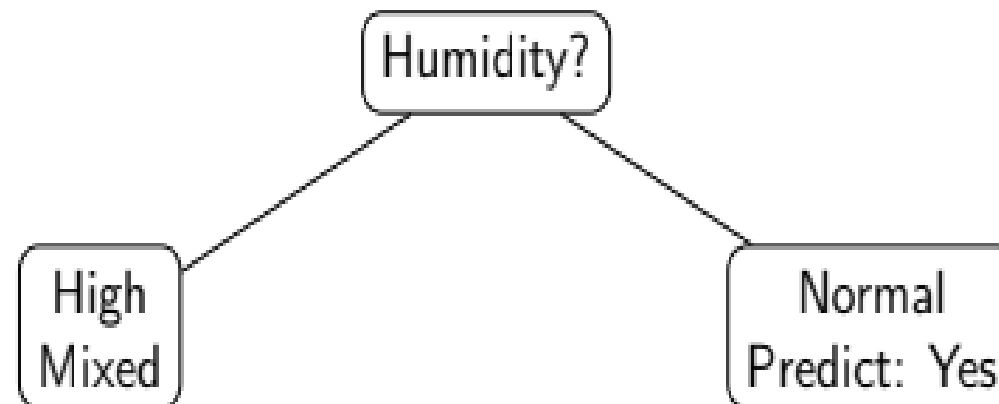
Since Humidity has lower Gini impurity, Humidity becomes the root node.

$$Root = Humidity$$

Interpretation

Humidity separates the classes better than Temperature in this bootstrapped dataset.

Dataset 1: Tree After First Split



Explanation

For Humidity = Normal:

$$\text{Yes} = 3, \quad \text{No} = 0$$

So this branch is pure and predicts Yes.

Next Step

The High branch is still mixed, so we split it again.

Dataset 1: High Humidity Branch

Inside the High branch, the examples are:

Day	Outlook	Temperature	Play Tennis
Day12	Overcast	Mild	Yes
Day14	Rain	Mild	No
Day2	Sunny	Hot	No

Now we test remaining features such as:

Outlook, Temperature

Goal

Choose the feature that gives the lowest Gini impurity in this branch.

Dataset 1: Split High Branch by Temperature

For Temperature inside the High branch:

$$\textit{Mild} : \textit{Yes} = 1, \textit{No} = 1$$

$$\textit{Hot} : \textit{Yes} = 0, \textit{No} = 1$$

$$\textit{Gini}(\textit{Mild}) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

$$\textit{Gini}(\textit{Hot}) = 1 - (0/1)^2 - (1/1)^2 = 0$$

Weighted Gini:

$$\textit{Gini}(\textit{Temperature}) = \frac{2}{3}(0.5) + \frac{1}{3}(0)$$

$$\textit{Gini}(\textit{Temperature}) = 0.333$$

Dataset 1: Split High Branch by Outlook

For Outlook inside the High branch:

Sunny : $Yes = 0, No = 1$

Overcast : $Yes = 1, No = 0$

Rain : $Yes = 0, No = 1$

Each branch is pure:

$$Gini(Sunny) = 0$$

$$Gini(Overcast) = 0$$

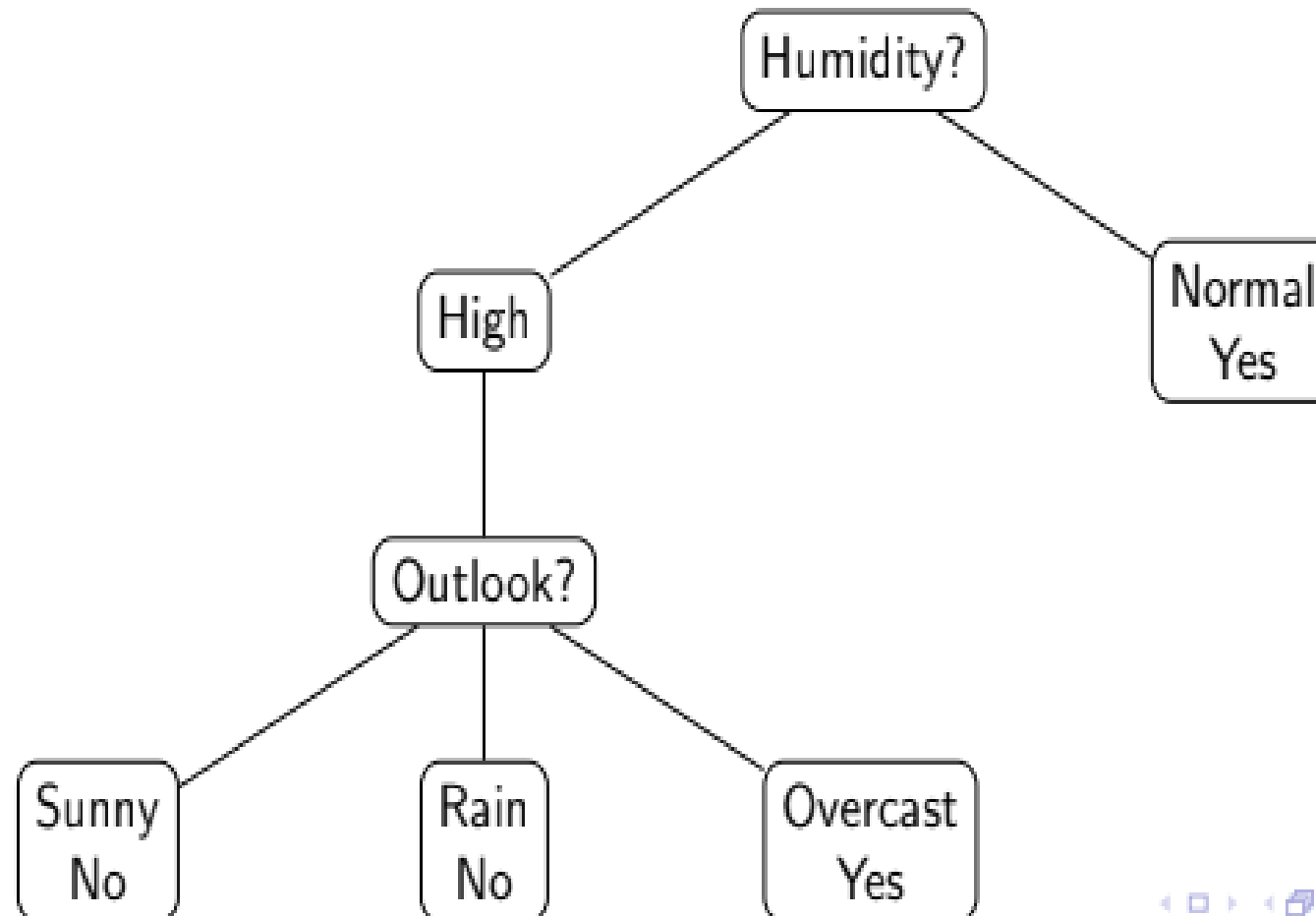
$$Gini(Rain) = 0$$

Dataset 1: Final Tree

Since:

$$Gini(Outlook) = 0 < Gini(Temperature) = 0.333$$

we choose Outlook inside the High branch.



Bootstrapped Dataset 2

Now another bootstrapped dataset is created:

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	No
Day2	Sunny	Hot	High	Strong	No
Day3	Overcast	Hot	High	Weak	Yes
Day4	Rain	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day2	Sunny	Hot	High	Strong	No

Suppose the random feature subset is:

Outlook, Temperature

Dataset 2: Selected Features

Day	Outlook	Temperature	Play Tennis
Day1	Sunny	Hot	No
Day2	Sunny	Hot	No
Day3	Overcast	Hot	Yes
Day4	Rain	Mild	Yes
Day5	Rain	Cool	Yes
Day2	Sunny	Hot	No

We compare:

Outlook and *Temperature*

Goal

Choose the root node using Gini impurity.

Dataset 2: Gini for Outlook

For Outlook:

Sunny : Yes = 0, No = 3

Overcast : Yes = 1, No = 0

Rain : Yes = 2, No = 0

All branches are pure:

$$Gini(Sunny) = 0, \quad Gini(Overcast) = 0, \quad Gini(Rain) = 0$$

Weighted Gini:

$$Gini(Outlook) = \frac{3}{6}(0) + \frac{1}{6}(0) + \frac{2}{6}(0)$$

$$Gini(Outlook) = 0$$

Dataset 2: Gini for Temperature

For Temperature:

Hot : *Yes* = 1, *No* = 3

Mild : *Yes* = 1, *No* = 0

Cool : *Yes* = 1, *No* = 0

$$Gini(Hot) = 1 - (1/4)^2 - (3/4)^2 = 0.375$$

$$Gini(Mild) = 0, \quad Gini(Cool) = 0$$

Weighted Gini:

$$Gini(Temperature) = \frac{4}{6}(0.375) + \frac{1}{6}(0) + \frac{1}{6}(0)$$

$$Gini(Temperature) = 0.25$$

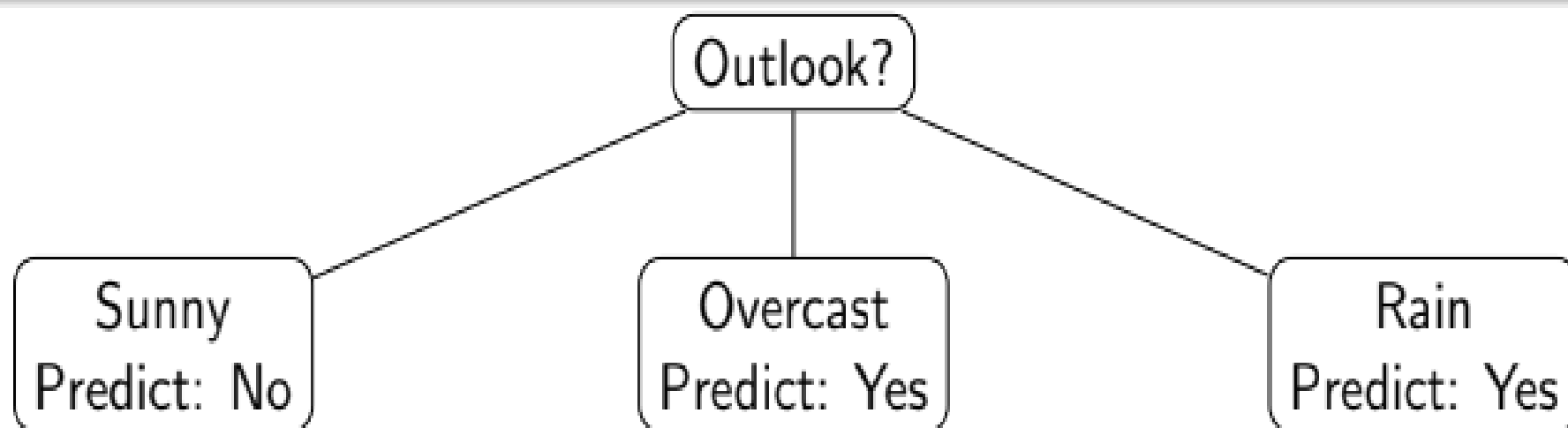
Dataset 2: Final Tree

Compare:

$$Gini(Outlook) = 0 \quad Gini(Temperature) = 0.25$$

Decision

Outlook has lower Gini impurity, so Outlook becomes the root.



This is Tree 2

This tree is different from Tree 1 because it used a different bootstrapped dataset and feature subset.

Bootstrapped Dataset 3

Now suppose a third bootstrapped dataset is created:

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day6	Rain	Cool	Normal	Strong	No
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	No
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day13	Overcast	Hot	Normal	Weak	Yes

Suppose the random feature subset is:

Humidity, Wind

Dataset 3: Selected Features

Day	Humidity	Wind	Play Tennis
Day6	Normal	Strong	No
Day7	Normal	Strong	Yes
Day8	High	Weak	No
Day9	Normal	Weak	Yes
Day10	Normal	Weak	Yes
Day13	Normal	Weak	Yes

We compare:

Humidity and *Wind*

Goal

Choose the root node using Gini impurity.

Dataset 3: Gini for Humidity

For Humidity:

High : Yes = 0, No = 1

Normal : Yes = 4, No = 1

$$Gini(High) = 1 - (0/1)^2 - (1/1)^2 = 0$$

$$Gini(Normal) = 1 - (4/5)^2 - (1/5)^2$$

$$Gini(Normal) = 1 - 0.64 - 0.04 = 0.32$$

Weighted Gini:

$$Gini(Humidity) = \frac{1}{6}(0) + \frac{5}{6}(0.32)$$

$$Gini(Humidity) = 0.27$$

Dataset 3: Gini for Wind

For Wind:

Strong : Yes = 1, No = 1

Weak : Yes = 3, No = 1

$$Gini(Strong) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

$$Gini(Weak) = 1 - (3/4)^2 - (1/4)^2 = 0.375$$

Weighted Gini:

$$Gini(Wind) = \frac{2}{6}(0.5) + \frac{4}{6}(0.375)$$

$$Gini(Wind) = 0.42$$

Dataset 3: Final Tree

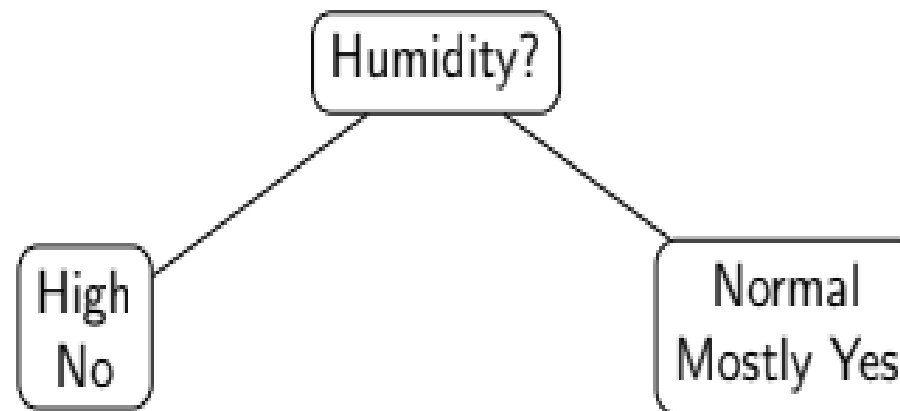
Compare:

$$Gini(Humidity) = 0.27$$

$$Gini(Wind) = 0.42$$

Decision

Humidity has lower Gini impurity, so Humidity becomes the root.



This is Tree 3

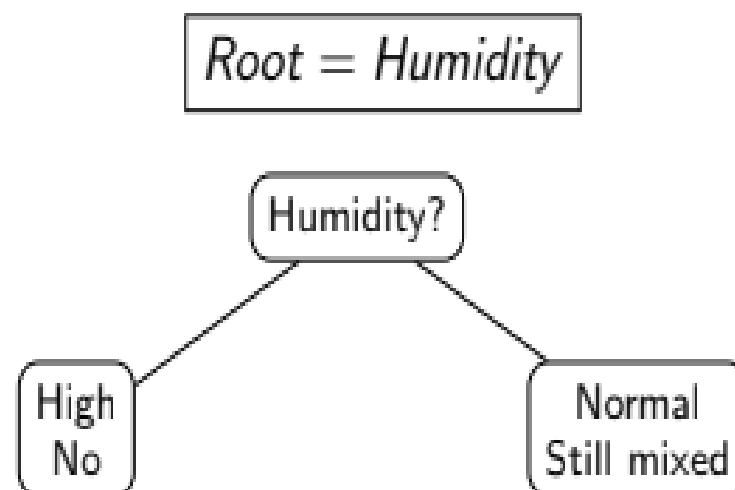
Dataset 3: Choose the Root

For Dataset 3, compare the Gini values:

$$\text{Gini}(\text{Humidity}) = 0.27 \quad \text{Gini}(\text{Wind}) = 0.42$$

Decision

Since Humidity has lower Gini impurity, Humidity becomes the root.



Dataset 3: Final Tree

Root:

Humidity

The Normal branch is not pure:

Humidity = Normal : Yes = 4, No = 1

So we split the Normal branch using Wind.

Condition	Result	Prediction
Humidity = High	Pure No	No
Humidity = Normal, Wind = Weak	Yes=3, No=0	Yes
Humidity = Normal, Wind = Strong	Yes=1, No=1	No

Teaching Point

We do not stop at “Mostly Yes” because the Normal branch is still mixed. We split it further using Wind.

Random Forest Prediction: Query

Now suppose we have a query:

Outlook	Temperature	Humidity	Wind	Play Tennis
Rain	Hot	High	Weak	?

Each tree gives a prediction.

Tree	Prediction
Tree 1	No
Tree 2	Yes
Tree 3	No

Majority Voting

The votes are:

Yes : 1

No : 2

Final Random Forest Decision

Since No has the majority vote:

Play Tennis = No

Important

One tree may be wrong, but the forest combines several trees to make a more stable decision.

Why Random Forest Works Better Than One Tree

Random Forest improves over a single decision tree because:

- Each tree sees a different bootstrapped dataset.
- Each tree uses a random subset of features.
- Errors from one tree can be corrected by other trees.
- Majority voting reduces instability.

Core Idea

Random Forest reduces overfitting by combining many diverse trees.

Random Forest and Feature Selection

Random Forest can also be used for feature selection.

Idea

A feature is important if it frequently helps reduce impurity across many trees.

For example, if **Humidity** is often selected near the root and reduces Gini impurity, then Humidity may be considered important.

Feature Importance

Random Forest can estimate which features are useful for prediction.

Missing Values using Random Forest

Random Forest can also help estimate missing values.

Suppose we have this dataset:

Chest Pain	Good Blood Circulation	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	??	??	No

Problem

Blocked Arteries and Weight are missing in the last row.

Initial Guess for Missing Values

Before using Random Forest, we first make simple initial guesses.

Categorical Missing Value

For Blocked Arteries, use the most frequent value.

Blocked Arteries = No

Numerical Missing Value

For Weight, use the average of known weights.

$$\text{Weight} = \frac{125 + 210}{2} = 167.5$$

Initial Completed Row

Yes, No, No, 167.5, No

Proximity Matrix Idea

After building a Random Forest, we pass every example through every tree.

Proximity

Two examples are considered close if they often end up in the same leaf node.

Example

If Example 3 and Example 4 end up in the same leaf in 8 out of 10 trees, their proximity is:

$$\frac{8}{10} = 0.8$$

Use

Examples with high proximity are used more strongly when estimating missing values.

Example Proximity Matrix

After running through 10 trees, suppose the proximity matrix is:

	Ex1	Ex2	Ex3	Ex4
Ex1	-	0.2	0.1	0.1
Ex2	0.2	-	0.1	0.1
Ex3	0.1	0.1	-	0.8
Ex4	0.1	0.1	0.8	-

Interpretation

Example 4 is closest to Example 3 because proximity is 0.8.

Estimating Missing Categorical Value

For the missing value **Blocked Arteries** in Example 4, use nearby examples.

Relevant proximities:

$Ex4$ with $Ex1 = 0.1$

$Ex4$ with $Ex2 = 0.1$

$Ex4$ with $Ex3 = 0.8$

Known values:

$Ex1 = No$, $Ex2 = Yes$, $Ex3 = No$

Since the largest proximity is with $Ex3$, and $Ex3$ has Blocked Arteries = No:

Blocked Arteries = No

Estimating Missing Numerical Value

For missing Weight in Example 4, use a weighted average.

Known weights:

$$Ex1 = 125, \quad Ex2 = 180, \quad Ex3 = 210$$

Proximities with Ex4:

$$0.1, \quad 0.1, \quad 0.8$$

Weighted average:

$$Weight = 125 \left(\frac{0.1}{0.1 + 0.1 + 0.8} \right) + 180 \left(\frac{0.1}{0.1 + 0.1 + 0.8} \right) + 210 \left(\frac{0.8}{0.1 + 0.1 + 0.8} \right)$$

$$Weight = 198.5$$

Estimated Weight

Updated Dataset After Missing Value Estimation

After estimating missing values:

Chest Pain	Good Blood Circulation	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	No	198.5	No

Iterative Idea

This process can be repeated until the missing value estimates become stable.

Random Forest: Full Workflow

- 1 Start with training data.
- 2 Create many bootstrapped datasets.
- 3 Select random feature subsets.
- 4 Train one decision tree on each dataset.
- 5 Use Gini impurity or another criterion for splitting.
- 6 For classification, combine tree outputs by majority vote.
- 7 For missing values, use proximity between samples.

Main Idea

Random Forest uses randomness plus many trees to make more reliable predictions.

Random Forest vs Single Decision Tree

Topic	Decision Tree	Random Forest
Number of trees	One	Many
Stability	Less stable	More stable
Overfitting risk	Higher	Lower
Interpretability	Very high	Moderate
Prediction	One tree decision	Voting/Averaging
Feature use	All features considered	Random subsets used

Trade-off

Random Forest is usually more accurate, but less directly interpretable than a single tree.

Advantages of Random Forest

- Works for classification and regression.
- Handles nonlinear relationships.
- Reduces overfitting compared with a single tree.
- Can estimate feature importance.
- Can help handle missing values.
- Usually gives strong performance without heavy tuning.

Reason

It combines many diverse decision trees.

Limitations of Random Forest

- Less interpretable than a single decision tree.
- Can be computationally expensive with many trees.
- Predictions are harder to explain step by step.
- Very large forests may require more memory.

Important

Random Forest improves accuracy and stability, but we lose some simplicity compared with one decision tree.

Summary

- Random Forest is an ensemble of decision trees.
- It uses bootstrapped datasets.
- It uses random subsets of features.
- Each tree is trained separately.
- For classification, final prediction is made by majority voting.
- Gini impurity can be used to build individual trees.
- Random Forest can also estimate feature importance and missing values.

Random Forest = Random Data + Random Features + Many Trees +
Voting

Thank You

Questions?